Journal of Intelligent & Fuzzy Systems 36 (2019) 775–786
DOI:10.3233/JIFS-18209
IOS Press

Bi-direction hierarchical LSTM with spatial-temporal attention for action recognition

Haodong Yang[a], Jun Zhang[a], Shuohao Li[a,*] and Tingjin Luo[b]
[a]*Science and Technology on Information Systems Engineering Laboratory, National University of Defense Technology, Changsha, China*
[b]*College of Science, National University of Defense Technology, Changsha, China*

Abstract. Human action recognition in naturalistic videos is an important task with a broad range of applications. Recently, the encoder-decoder framework based on attention mechanism has been applied to action recognition. Although such conventional methods reach state-of-the-art, they always face a bottleneck of distinguishing similar actions. To solve this problem, we propose a novel recurrent attention convolutional neural network (RACNN), which incorporates convolutional neural networks (CNNs), long short-term memory (LSTM) and attention mechanism. Inspired by the composition of the action, the pre-action and the result of action might be important parts of an action, we introduce bi-direction LSTM with hierarchical structure. Additionally, the separated spatial-temporal attention is employed into our method. Furthermore, we find that incorporating spatio-temporal features extracted from three-dimensional CNNs (3DCNNs) and RGB features can enhance the relationship mined in each frame. Our comprehensive experimental results on two benchmark datasets, i.e., HMDB51 and UCF101, verify the effectiveness of our proposed methods and show that our proposals can significantly outperform the current state-of-the-art methods.

Keywords: Action recognition, bi-direction hierarchical LSTM, spatial-temporal attention

1. Introduction

Human action recognition in videos is one of fundamental challenging tasks in computer vision. In literature, it extends over a broad range of high-impact societal applications from video surveillance to human-computer interaction, web-video search, retrieval and quality-of-life improvement for elderly care. Moreover, it plays an important role in video summary and understanding. In the Oxford Dictionary, action is defined as "the fact or process of doing something, typically to achieve an aim", and activity is "a thing that a person or group does or has done". S. Herath et al. [1] provide a consolidated definition that "Action is the most elementary human surrounding interaction with a meaning." Generally speaking, action recognition aims at categorizing the actions or behaviors of one or more persons in a video sequence.

Nevertheless, due to the diverse sets of objects, scenes, actions, attributes and salient contents in video, it is difficult to extract appropriate spatial and temporal features by effectively modeling the evolutions of different actions. With the significant advancement of deep neural networks (DNNs) in computer vision, many studies have adopted CNNs to extract features. For example, Sharma et al. [2] focus on the recognition from RGB data and employ CNNs, which have good ability on producing rich

*Corresponding author. Jun Zhang Science and Technology on Information Systems Engineering Laboratory, National University of Defense Technology, Changsha, 410073, China. E-mail:zhangjun1975@nudt.edu.cn.

ISSN 1064-1246/19/$35.00 © 2019 – IOS Press and the authors. All rights reserved

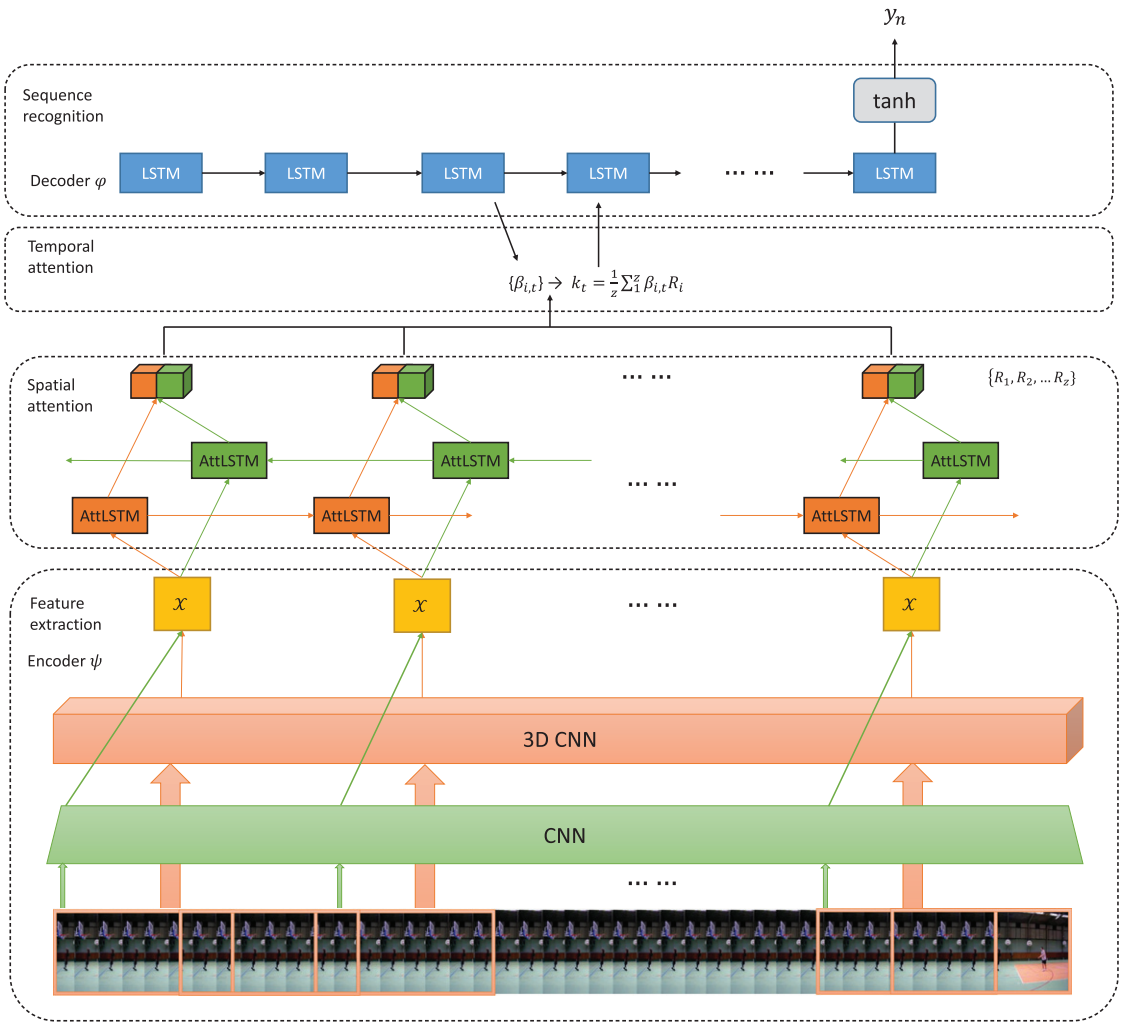

Fig. 1. The architecture of the proposed RACNN model. It contain four major components: (1) convolutional feature extraction; (2) bi-direction LSTM sequence model and spatial attention model; (3) temporal attention model; and (4) sequence recognition. CNNs encode the input video frames into feature sequences. AttLSTM means hierarchical LSTM with spatial attention model which would be detailedly shown in next subsection. The output of bi-direction AttLSTM is designed to be integrated and sent to temporal attention model. After refining the input, our architecture would take the output of the last LSTM unit and calculate result.

representation and achieve promising results on several challenging tasks, to improve the performance of recognition. However, the temporal attribute of video cannot also be neglected because of the continuity of action. The methods in [2] only use RGB data and lose meaningful information of the 3D space and the flexibility of achieving human location and scale invariance. To solve this problem, the direct approach for action recognition is to arm the convolutional operation with temporal information such as 3DCNNs, which are the natural extension of a standard two-dimensional convolutional neural network (2DCNNs) to cover the temporal domain as a third dimension. 3DCNNs, which are firstly proposed by Ji et al. [3], are more effective than 2DCNNs with 2D convolutional kernels [4]. Recently, Tran et al. [5] propose C3D to solve action recognition problem and obtain a good performance. Meanwhile, to get spatio-temporal features, Simonyan and Feichtenhofer et al. [6, 7] propose a two-stream architecture to learn 2D filters for the optical flow and appearance variations independently. Even, the current state-of-the-art method [8], uses a variant of discriminatively trained 2DCNNs over the appearance (frames) and in some cases motion (optical flow) modalities of the input video. These fusion methods outperform

the work in [5], which have only explored relatively shallow 3D architectures.

CNNs consider appearance features frame by frame instead of interactions across the temporal domain. In order to model these dynamics between frames, recurrent neural networks (RNNs) seem like an excellent candidate due to its sequence modeling capabilities. In the variant LSTM [9] networks, a gating mechanism over an internal memory cell learns long-term and short-term dependencies in the sequential input data. The results show that these methods perform well in the domain of speech recognition, machine translation, and video description [10–12]. Moreover, they have also started picking up momentum in action recognition [2]. Although at first glance LSTMs seem ideal, they face certain shortcomings. LSTMs treats all incoming data as vectors, which leads to lack of spatial information, even if they have a spatial structure. To tackle these issues, attention mechanism could vectorize the features with spatial structure into vectors. In modelling videos, the experiments in [13] show that attention must be considered. Hence, they hardwire the LSTM network with spatio-temporal attentions. Recently, other studies [2, 14, 15] show that models of visual attention may help extract important cues on small or far away objects (or people) and have drawn considerable interests.

For some actions with significant differences, e.g., 'stand' versus 'sit', these methods have the capacity to differentiate them; while for some similar actions, they find it difficult to classify them. For example, in HMDB51 dataset, action 'jump' have many similar parts of the action 'run', in another word, the 'jump' consists of a little 'run'. Inspired by the composition of action, the pre-action and the result of action might be important parts of an action, we introduce bi-direction LSTM with hierarchical structure [16] to for incorporating global information for the action. Additionally, we take the separate spatio-temporal attention into consideration to mine the relationship in frame and refine the input features representatively. This framework is also named as RACNN which is similar to the work of [2] and [13]. In our framework, we consider the above analyses and extract spatio-temporal features from 3DCNNs. However, these features might be inferior to RGB features for mining the relationship in each frame, thus, we add the RGB data which is vectorized by spatial attention to gain the spatial relations in frame. By this way, we can represent the spatio-temporal information in video effectively. Finally, the total architecture is shown in Fig. 1 and the details of each part will be described in section 3.

In summary, the contributions of proposals can be concluded as: 1) We introduce a novel deep learning framework for action recognition in videos which combine the RGB feature with 3D spatio-temporal feature; 2) We integrate separate spatial and temporal attention model into our framework. Spatial attention is utilized to mine the relationship in each frame to augment spatial relations in frames, while temporal attention learns to pick up the key features for decoder; 3) We conduct an extensive set of experiments to demonstrate that our proposals are superior to state-of-the-art methods on two benchmark datasets: HMDB-51 [17] and UCF-101 [18].

The rest of the paper is organized as followed: In section 2, we review the related work. We prefer to present the development of action recognition and show some state-of-the-art works. The proposed methods and the principle of our method are presented in section 3. Experimental results and some training details are discussed in section 4. Finally, section 5 makes a conclusion of the whole work.

2. Related work

Human action recognition is a longstanding topic in computer vision community and is well studied with various standard benchmarks. The literature on action recognition in video is vast and too broad for us to cover here completely. In this work, we present some related works on ConvNet architecture, LSTM-like architecture and attention mechanism.

2.1. ConvNet architecture

Compared with hand-craft shallow features, CNNs try to learn the representation of video automatically from large scale labeled video data and have been proven more efficient for representing a video. For example, Karpathy et al. [19] directly apply CNNs to multiple frames in each sequence and obtain the temporal relations by pooling using single, late, early and slow fusion. Nevertheless, the results of this scheme are just marginally better than those of a single frame baseline, and indicate that motion features are difficult to obtain by simply and directly pooling spatial features from CNNs. To ameliorate this shortcoming, Simonyan et al. [6] propose a two-stream ConvNet architecture, which incorporates spatial and temporal networks trained on multi-frame dense optical

flow, could be able to achieve very good performance. Furthermore, Simonyan and Feichtenhofer et al. [6, 7] introduce multi-stream networks which split the input video into multiple modalities such as RGB, optical flow, warped flow etc, train standard image based CNNs on top of those, and late-fuse the predictions from each of the CNNs. Based on this baseline, Limin et al. [8] propose a novel framework, temporal segment network (TSN), inspired from long-range temporal structure modeling, and obtain the state-of-the-art performance on the datasets of HMDB51 and UCF101. Another paradigm of action recognition, 3DCNNs, when solving above issues receive much attention from the community, regard the video as a spatio-temporal blobs and train 3D filters to recognizing action. 3DCNNs are firstly introduced by Ji et al. [3] and learn 3D convolution kernels in both spatial and temporal space based on a straightforward extension of the established 2DCNNs. However, the performance of this method has been harder to scale and multi-stream methods. While conventional researches just employ shallow 3D architectures, thus, Hara et al. [20] examine the architectures of various 3DCNNs from relatively shallow to very deep ones on current video datasets HMDB51 and UCF101, and their 3D architectures outperform state-of-the-art multi-stream model. Our approach is complementary to these paradigms and extracts the 3D features by this architecture.

2.2. LSTM-like architecture

RNNs have great capabilities of modeling dynamics between frames. Especially, LSTM networks avoid gradient vanishing or exploding during back forward propagation due to its gating mechanism and have been employed in nature language process, machine translation and other sequence problems [21–23]. Most recent activity recognition [2, 24–27] have CNNs underlying the LSTMs to mine the information in the feature map. Recently, Ng et al. [25] and Donahue et al. [24] propose LSTMs that explicitly model short snippets of ConvNet activations. Ng et al. [25] have proven that the average fusion of LSTM networks with appearance and flow input improves over improved dense trajectories [28] and the baseline two-stream approach [6]. To make model better performance, they pre-train their architecture on 1 million sports videos. Srivastava et al. [2] also pre-train on hundreds of hours of sports video, but they propose an interesting LSTM based unsupervised training method and then fine-tune this unsupervised pre-training LSTM to adopt human action recognition tasks. More importantly, they use an encoder LSTM to map an input sequence into a fixed-length representation, and use single or multiple decoder LSTMs to perform reconstruction of the input sequence or prediction of the future sequence. Donahue et al. [24] propose a class of end-to-end trainable CNN and RNN architectures to handle different kinds of inputs. However, Sun et al. [26] reckon that the traditional LSTMs with matrix multiplications are adopted without exploiting the spatial correlations in the video frames and directly applying the LSTM models to videos based action recognition is not satisfactory since the spatial correlations and motion dynamics between the frames are not well presented. Thus, they propose Lattice-LSTM (L2STM), to extends LSTM by learning independent hidden state transitions of memory cells for individual spatial locations. In [27], they introduce deep bidirectional LSTM network with stack multiple LSTM layers in both forward pass and backward pass, and it is capable to learning long term sequences and process lengthy videos by analyzing features for a certain time interval.

2.3. Attention mechanism

Theoretically, human perception focuses selectively on parts of the scene to acquire information at specific places and times [29]. Attention mechanism solves these problems and has great achievement in image/video caption, scene text recognition and action recognition [2, 30, 31]. Attention mechanism is usually categorized into two classes: hard attention and soft attention. Hard attention takes hard decisions when choosing parts of the input data; while, soft attention takes the entire input into account, weighting each part of the observations dynamically. Where, Sharma et al. [2] propose the soft attention with LSTM to integrate convolutional features from different parts of a space-time volume for action recognition from RGB data and compare the performance between hard attention and soft attention. Xu et al. [30] propose soft-attention architecture for image captioning and receive a great performance. However, they completely ignore the motions in videos. Furthermore, Yeung et al. [32] report a temporal recurrent attention model for dense labelling of videos. They do dense action labeling using a temporal attention based model on the input-output context and report higher accuracy and better understanding of temporal relationships in action videos. In our

work, we propose separate spatial and temporal attention networks for action recognition. At each video frame, the spatial attention model gives more importance to the joints most relevant to the current action, whereas the temporal model selects the frames.

3. Methodology

In this section, we firstly describe basic LSTM as the decoder for action recognition task. Then we introduce our encoder-decoder framework, and we expound the main component of our framework in detail: 1) Feature extracted from CNNs; 2) Bi-direction hierarchical LSTM; 3) Spatial-Temporal attention; 4) Sequence recognition.

3.1. A basic LSTM for action recognition

In order to model the dynamics between frames, we take LSTMs which have started picking up momentum in action recognition into consideration. What is more, recent advances [2, 33] in computer vision also suggest that LSTMs have potential to model videos for action recognition.

LSTM units. LSTMs are recurrent modules who have the capability of learning long-term dependencies and use a hidden state augmented with nonlinear mechanisms to allow the state to propagate without modification. A basic LSTM consists of three gates, and a single memory cell c_t. Due to LSTMs use multiplicative gates to control access to the error signal propagating through time, thus, they alleviate the short memory in RNNs. Basically, the memory cell and gates in a LSTM block are defined as follows:

$$\begin{aligned} i_t &= \sigma(W_{ix}x_t + W_{im}h_{t-1} + b_i) \\ f_t &= \sigma(W_{fx}x_t + W_{fm}h_{t-1} + b_f) \\ o_t &= \sigma(W_{ox}x_t + W_{om}h_{t-1} + b_o) \\ g_t &= \tanh(W_{gx}x_t + W_{gm}h_{t-1} + b_g) \\ c_t &= f_t \odot c_{t-1} + i_t \odot g_t \\ h_t &= o_t \odot \tanh(c_t) \end{aligned} \tag{1}$$

Where i_t is the input gate, f_t is the forget gate, o_t is the output gate, g_t is calculated as shown in Equation 1. c_t is the memory cell state at t, h_t is the hidden state at t, and x_t stands for input features at t. Speaking meaningfully, i_t allows incoming signals to alter the state of the memory cell or block it; f_t controls what to be remembered or to be forgotten by the cell, and somehow can avoid the gradient from vanishing or exploding when back propagating through time. Finally, o_t allows the state of the memory cell to have an effect on other neurons or prevent it.

The weight matrices are denoted by W_{ij} and biases b_j, which are the trainable parameters. σ represents the logistic sigmoid nonlinear activation function mapping real numbers to (0,1), and it can be thought as knobs that LSTM learns to selectively forget its memory or accept the current input. tanh denotes the hyperbolic tangent function. $\odot$ is the element-wise product with the gate value. For convenience, we denote $h_t, c_t = LSTM(x_t, h_{t-1}, c_{t-1})$ as the computation function for updating the LSTM internal state.

Action Recognition. Given a video input $\mathcal{V}$, the encoder network encodes it into a series of feature vectors:

$$V = \{v_1, v_2, \ldots, v_n\} = \phi(\mathcal{V}) \tag{2}$$

where $\phi(x)$ usually denotes CNN networks, n denotes the number of frames in $\mathcal{V}$, $v_i \in R^d$ is the frame-level feature of the i-th frame, which has D-dimension.

$$\begin{aligned} h_t, c_t &= LSTM(v_t, h_{t-1}, c_{t-1}) \\ y_t &\sim argmaxSoftmax(W_y h_t + b_y) \end{aligned} \tag{3}$$

After generating result from every LSTM units, the main methods make a softmax among all outputs. Then we can obtain the final category.

3.2. Convolutional features extracted from CNNs

To date, generating rich representations of the input images with CNNs has shown great success in the process of image classification, object detection and image captioning. In many tasks, it is natural to use CNNs as the encoder to extract the convolutional features. Moreover, in order to obtain the spatio-temporal features, 3DCNNs also receive much attention.

In our work, we extract RGB features, saptio-temporal features from 2DCNNs and 3DCNNs respectively. For 2DCNNs, networks which are pre-trained on ImageNet dataset are employed in our work to extract appearance features. In general, our architecture can adopt any deep convolution networks for feature extraction. Where, as we known, resnet [34] has won the first prize in the imageNet

Table 1
Architecture of Resnet-101,152. Residual blocks are shown in brackets. Each convolutional layer is followed by batch normalization and ReLU. Down-sampling is performed by conv3_1, conv4_1, conv5_1 with a stride of 2. T stands for temporal, and S represents for spatial

Layer Name	101-layer	152-layer
Conv1	$7 \times 7 \times 7$, 64, stride 1 (T), 2(S)	
Conv2	$\begin{Bmatrix} 3 \times 3 \times 3, & 64 \\ 3 \times 3 \times 3, & 64 \end{Bmatrix} X3$	$\begin{Bmatrix} 3 \times 3 \times 3, & 64 \\ 3 \times 3 \times 3, & 64 \end{Bmatrix} X3$
Conv3	$\begin{Bmatrix} 3 \times 3 \times 3, & 128 \\ 3 \times 3 \times 3, & 128 \end{Bmatrix} X4$	$\begin{Bmatrix} 3 \times 3 \times 3, & 128 \\ 3 \times 3 \times 3, & 128 \end{Bmatrix} X8$
Conv4	$\begin{Bmatrix} 3 \times 3 \times 3, & 256 \\ 3 \times 3 \times 3, & 256 \end{Bmatrix} X23$	$\begin{Bmatrix} 3 \times 3 \times 3, & 256 \\ 3 \times 3 \times 3, & 256 \end{Bmatrix} X36$
Conv5	$\begin{Bmatrix} 3 \times 3 \times 3, & 512 \\ 3 \times 3 \times 3, & 512 \end{Bmatrix} X3$	$\begin{Bmatrix} 3 \times 3 \times 3, & 512 \\ 3 \times 3 \times 3, & 512 \end{Bmatrix} X3$
global average pool, C-d fully-connected, softmax		

competition. And these networks have a great ability to represent features of ImageNet dataset, thus, we make a use of resnet-152 to extract features. In the following subsection, we evaluate the performance of different networks. To understand structure of resnet clearly, we compare the architecture of resnet-101 and resnet-152, and the parameter of these networks shown in the Table 1.

First of all, we employ opencv toolkit to split each video into a list of frames, and then, we pick up the some of which with fixed interval. Following that, we extract feature maps before the last convolutional layer, aka, block4_unit3_conv2, which have dimension of $D \times K \times K$ ($512 \times 7 \times 7$ used in resnet-152), where $K \times K$ is the size of feature map and D is the number of the feature dimensions. In another word, at each time step t, we extract $K^2 D$-dimensional vectors. We refer to these vectors as feature slices into a feature cube, and denoted as:

$$\begin{aligned} X_t &= (X_{t,1}, X_{t,2}, \cdots, X_{t,K^2}) \\ X_{t,i} &\in R^D \end{aligned} \tag{4}$$

Where each vertical features mean that the same region of feature map in different dimensions. At time step t, these K^2 vertical features map to different overlapping regions in the input space and our model chooses to focus its attention on these K^2 regions.

For 3DCNNs, in the stage of spring up, the shallow network was adopted in the architecture. This weakness results in receiving a marginally worse performance than two-stream architecture. We take resnet-based architectures with 3D convolutional filter pre-trained on kinetics dataset and shown in Fig. 2. In our work, we take resnext-101 block as

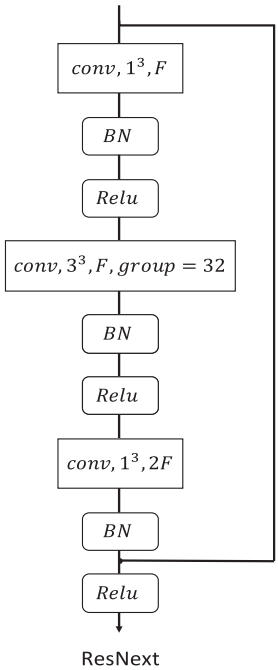

Fig. 2. Block of ResNext architecture. We represent $conv, x^3, F$ as the kernel size, and the number of feature maps of the convolutional filter are $x \times x \times x$ and F, respectively, and group as the number of groups of group convolutions, which divide the feature maps into small groups. *BN* refers to batch normalization. *Relu* stands for activate function. Shortcut connections of the architectures are summation.

spatio-temporal features extractor. Then, we split each video into a number of video clips (the number of video clips is set as 16). Through features extractor, we obtain fixed-length features.

3.3. Bi-direction hierarchical LSTM

In this subsection, we will divide our bi-direction hierarchical LSTM and describe every part.

3.3.1. Hierarchical LSTM

Inspired by our previous work [16], we introduce hierarchical LSTM. Our hierarchical LSTM integrates two layers of LSTM. The bottom LSTM layer is used to efficiently decode visual features. As for appearance feature, the bottom LSTM layer is used to mine the relationship in each frame; some actions might require paying close attention to objects being interacted by the human, like in case of 'drinking from mug' versus 'drinking from water bottle'. Thus, spatial attention model used to gain the relationship. Meanwhile, the top LSTM is mainly focusing on mining context information for activities.

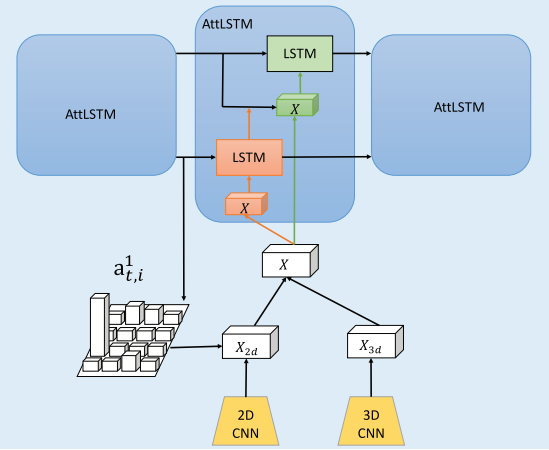

Fig. 3. This hierarchical architecture of 'AttLSTM' which shown in the Fig. 1. X_{2d} stands for the features extracted from 2D CNNs and X_{3d} represents the features extracted from 3D CNNs. $\alpha^1_{t,i}$ is the weights generated from previous AttLSTM.

In another word, top LSTM get more global information and previous unit would instruct the next unit. For instance, some actions might have similar background or the preparing activities, like 'jump' versus 'run'; the 'jump' usually contain a part of 'run', it is easy to be confused. However, the hierarchical structure might obtain comprehensive information from features. In Fig. 3, we will show the hierarchical structure.

Additional, we also incorporate spatial attention mechanisms into this part (see section 3.4.1). The bottom LSTM is the same as the ordinary LSTM, the hidden state h_t generated from the current input x_t, previous hidden state h_{t-1} and the memory state c_{t-1}:

$$h_t, c_t = LSTM(x_t, h_{t-1}, c_{t-1}) \tag{5}$$

In order to converge faster, we use the following initialization strategy (see Xu et al. [30]) for the cell state and the hidden state of the LSTM:

$$\begin{aligned} c_0 &= f_{init,c}(\frac{1}{T}\sum_{t=1}^{T}(\frac{1}{K^2}\sum_{i=1}^{K^2}X_{i,t})) \\ h_0 &= f_{init,h}(\frac{1}{T}\sum_{t=1}^{T}(\frac{1}{K^2}\sum_{i=1}^{K^2}X_{i,t})) \end{aligned} \tag{6}$$

The top LSTM would contain not only the previous hidden state and input but also the bottom hidden state. Drill down, both previous and bottom hidden states are facilitated to guide the input. To reduce the computation complexity of LSTM, we mix them up by adopting variant spatial attention (see section. 3.4.1). Consequently, the top LSTM is also the same as the ordinary LSTM.

3.3.2. Bi-direction hierarchical LSTM

To take the integrity of action into consideration, we introduce bi-direction (BiLSTM) into our architecture. In BiLSTM, the output at time t is not only dependent on the previous frames in the sequence, but also on the upcoming frames. Bidirectional RNNs are quite simple, having two RNNs stacked on top of each other. One RNN goes in the forward direction and another one goes in the backward direction. The combined output is then computed based on the hidden state of both RNNs. In our work, we are using multiple LSTM layers, so our scheme has two LSTM layers for both forward pass and backward pass. Where 'fw' is forward pass and 'bw' is backward pass. Both 'fw' and 'bw' consist of two LSTM cells, making our model a deep bidirectional LSTM. The output of BiLSTM is denoted as:

$$\begin{aligned} R_{fw,t} &= LSTM_{fw}(h_{t-1}, x_t) \\ R_{bw,t} &= LSTM_{bw}(h_{t-1}, x_t) \\ R_t &= f(R_{fw,t}, R_{bw,t}) \\ R &= \{R_1, R_2, \cdots, R_z\} \end{aligned} \tag{7}$$

where $f(x)$ is used to mix two output into together, z is the number of LSTM units in each LSTM layers.

3.4. Spatial-temporal attention

A lot of works [2, 30] gain the great achievement with attention mechanism, and they have proved that soft attention could gain the performance in CNN-RNN model for action recognition task. In above section, we have mentioned that attention mechanism would be used to mine relationship in each frame. Besides that, we also propose temporal attention to gain temporal information. In this part, we describe our attention in detail.

3.4.1. Spatial attention model

Obviously, it is naturally believed that regions of frame have the different weight and are partly irrelevant to action. Thus, in order to improve the accuracy of our model, we pay more attention to the regions where the action is happening.

In the bottom LSTM layer, we design a function that takes h_{t-1} and x as the input, then that returns

the unnormalized relevance score at time step, which can be shown in below:

$$e^1_{t,i} = W_1^T \tanh(W_{1,a}h_{1,t-1} + U_{1,a}x_{t,i} + b_{1,a}) \quad (8)$$

Once the relevance scores for feature sequence are all computed, we normalize them to obtain the $a^1_{t,i}$:

$$a^1_{t,i} = \frac{exp(e^1_{t,i})}{\sum_{j=1}^{K\times K} exp(e^1_{t,j})} \qquad j \in 1...K^2 \quad (9)$$

The K^2 vectors in the appearance feature cubes correspond to K^2 regions in the t-th frame and our model chooses to focus its attention on these K^2 regions. $a^1_{t,i}$ is a softmax over $K \times K$ locations and denotes weight of the i-th region. At each time step, we update the parameters of attention model, and get the input x^1_t by following function:

$$x^1_t = \sum_{i=1}^{K\times K} X_{t,i}a^1_{t,i} \quad (10)$$

While the input parameter in top LSTM is more than the bottom one. Thus, to make the two LSTM have the same structure, we unify the input through different attention mechanism:

$$\begin{aligned} c_{t,i} &= \tanh(W_{2,pa}h_{1,t} + W_{2,a}h_{2,t-1} + U_{2,a}x_{t,i} + b_{2,a}) \\ e^2_{t,i} &= W_2^T c_{t,i} \end{aligned} \quad (11)$$

Similar to Equation 9, we normalize relevance score at time step and get $a^t_{2,i}$

$$a^2_{t,i} = \frac{exp(e^2_{t,i})}{\sum_{j=1}^{K\times K} exp(e^1_{t,j})} \qquad j \in 1...K^2 \quad (12)$$

where $h_{1,t}$ is hidden state generated from first attention layer at time t, $h_{2,t-1}$ represents hidden state generated from second attention layer at time $t-1$. In Equation 11 the W_w^T, $W_{2,pa}$, $W_{2,a}$, $U_{2,a}$ and $b_{2,a}$ are estimated together in model training procedure. After that, we could get the input of next LSTM, which is determined by the function as Equation 10:

$$x^2_t = \sum_{i=1}^{K\times K} X_{t,i}a^2_{t,j} \quad (13)$$

3.4.2. Temporal attention model

As shown in Fig. 1, we will adopt this temporal attention mechanism among the output of BiLSTM. Since the information in video is redundant, temporal relationship of video also play an important role in recognizing action. This strategy is similar to mean every input features, in our work, we do not compute the average of features. Instead of using a simple average strategy, we want to take the dynamic weight sum of the temporal feature vectors according to attention weights $\beta_{i,t}$, which are calculated by a soft attention:

$$k_t = \frac{1}{n}\sum_{i=1}^{z} \beta_{i,t}R_i \quad (14)$$

where $\beta_{i,t}$ generated from the next stage.

3.5. Sequence recognition

Grave et al. [35] has shown that it is natural to recognize sequence data with RNN. Above steps, we encode the features extracted from CNNs and process features via loop processor. In this stage, we use one layer LSTM to decode sequence, and recognize action. Different from popular methods, we only take the output of the last LSTM unit as our final result. Thus, the decoder function can be depicted as following:

$$\begin{aligned} h_t, c_t &= LSTM(k_t, h_{t-1}, c_{t-1}) \\ y &\sim (Wh_n + b) \end{aligned} \quad (15)$$

where W and b will be learned in network.

4. Experimental evaluation

4.1. Experimental dataSet

To verify the effectiveness of our methods, we conduct experiments mainly on two public action recognition benchmark datasets, namely UCF-101 and HMDB-51, which are currently the largest and the most challenging annotated action recognition datasets.

The HMDB51 dataset is a large collection of realistic videos from various sources, including movies and web videos. It is composed of 6,766 video clips from 51 action categories, with each category containing at least 100 clips. These clips are labeled with 51 classes of human actions like Clap, Drink, Hug, Jump, Somersault, Throw and many others. This dataset is more challenging than others because it contains scenes with more complex backgrounds. It has similar scenes in different categories, and it has a small number of training videos. HMDB-51 have

three split settings to separate the dataset into training and testing videos. Our experiments follow the original evaluation scheme, but only adopt the first training/testing split. In this split, each action class has 70 clips for training and 30 clips for testing.

The UCF101 dataset is composed of about 13,320 realistic user-uploaded video clips and 101 action categories, with each category containing at least 100 clips. The database is particularly interesting, because it gives the largest diversity in terms of actions and with the presence of large variations in camera motion, object appearance and pose, object scale, viewpoint, cluttered background, illumination conditions, etc, and it is the most challenging data set to date. There are 3 splits for training and testing (70% training and 30% testing). We tested our model on the first training/testing split in the experiments. Classification accuracy is used as evaluation measure.

4.2. Implementation details

4.2.1. Experimental environment

Our experiments are carried out on a workstation with a 4GHz Intel(R) Core (M) i7-4790 CPU, 64G RAM and an NVIDIA(R) GeForce 1080ti GPU. In the process of training and testing, our model uses CUDA to accelerate the experimental process.

4.2.2. Prepare process

For RGB data, our architecture can adopt any deep convolution networks for feature extraction, therefore, we firstly split videos into frames and resized to 224×224 resolution. Because the input dimension of LSTM is fixed, we pick up 16 video frames with interval. When comes to 3DCNNs, we employ the best model tested in [20] into our architecture, where they set each video clip as 16 frames. Thus, in order to be compatible with the input dimension of RGB, we extract 16 clips in each video. Here, sliding window method used in splitting video.

4.2.3. Training details

We implement proposed architecture based on the TensorFlow [36] framework. When constructing the network, LSTM is taken as recurrent processor in both spatial attention layer and sequence recognition layer. Especially, spatial and temporal attention models are embedded into the LSTM unit. Moreover, the dimensionality of the LSTM hidden state, cell state, and the hidden layer were set to 1024. For output layer, we take the ouput of the last one LSTM unit as our final result. In order to avoid overfitting, we use dropout (in [33]) of 0.5 at all non-recurrent connections. After that, we employ Adam optimization algorithm to train all models. Next step, the learning rate starts from 0.006, then we use exponential decay algorithm which was implied in TensorFlow toolbox and the decay rate is set to 0.9.

Table 2
CNNs encoder analysis on first split of HMDB51 and UCF101 with RGB data. Classification accuracy is used as evaluation measure which represented as recognition accuracies (%). The **bold** text represents the best result.

Model	HMDB51	UCF101
GoogleNet	54.6	87.9
Vgg	55.2	88.5
ResNet-101	55.9	89.4
ResNet-152	**56.4**	**89.8**

4.3. Evaluation of proposed method

4.3.1. The effect of different CNN encoders

In the famous imageNet competition, resnet has get great performance due to deep structure. Since action recognition is not the same as image classification, we verify the result of different CNN encoders. To date, there are 4 widely used CNN encoders including GoogleNet, Vgg, ResNet-101 and ResNet-152 to extract visual features. In this sub-experiment, we study the influence of different versions of CNN encoders on our framework. The experiments are conducted on RGB data in the first split of HMDB51 and UCF101 datasets and the results are shown in Tab. 2. These above networks are all pre-trained on imageNet dataset. We can easily find that by taking ResNet-152 as the visual decoder, our method perform best with 56.4% on HMDB51 and 89.8% on UCF101.

The 3DCNNs used in our architecture is taken from [20], they have proved that ResNeXt-101 (64f) has the best performance in recognizing actions.

4.3.2. Architecture exploration and comparison

In this sub-experiment, we explore the impact of three proposed components, including basic LSTM proposed in section. 1 (basic LSTM), BiLSTM with hierarchical structure, hierarchical BiLSTM with spatial attention and the proposed framework. In order to conduct a fair comparison, all the methods take ResNet-152 as the encoder. We conduct the same experiments on the first split of HMDB51 datasets. We take appearance and spatio-temporal features into comparison. The experimental results are shown in Tab. 3 . It shows that our proposed method achieves the best results in all features.

Table 3
Performance of different components on first split of HMDB51 with both RGB and 3D features. Classification accuracy is used as evaluation measure which represented as recognition accuracies (%). The **bold** text represents the best result

Method	RGB	3D	RGB+3D
Basic LSTM	54.6	69.4	70.1
Hierarchical BiLSTM	55.2	69.8	70.6
Hierarchical BiLSTM + ATT(S)	56.1	70.1	71.4
Hierarchical BiLSTM + ATT(S,T)	**56.4**	**70.6**	**71.9**

Table 4
Comparison of our proposed method to the state-of-the-art methods on HMDB51 and UCF101. Accuracy (%) is mean accuracy among three splits. The **bold** text represents the best result

Model	Method	HMDB51	UCF101
Traditional	iDT+FV [28]	57.2	85.9
	iDT+HSV [40]	61.1	87.9
	VideoDarwin [41]	63.7	-
	MPR [42]	63.5	-
Deep	EMV-CNN [43]	-	86.4
	Two stream [6]	59.4	88.0
	$F_{ST}CN$ [37]	59.1	88.1
Very Deep	C3D [5]	-	85.2
	VideoLSTM [13]	56.4	89.2
	TDD+FV [38]	63.2	90.3
	Fusion [7]	65.4	92.5
	L^2STM [26]	66.2	93.6
Complex	ST-ResNet [39]	66.4	93.4
	TSN [8]	69.4	94.2
Our proposed method	**71.9**	**94.8**	

4.3.3. *Compare with the-state-of-the-art methods*

The state-of-the-art algorithm comparison is presented in Tab. 4, where we compare our method with traditional approaches such as improved trajectories (iDTs) [28] and deep learning architectures such as two-stream networks [6], factorized spatio-temporal convolutional networks (FSTCN) [37], 3D convolutional networks (C3D) [5], trajectory-pooled deep convolutional descriptors (TDD) [38], and spatio-temporal fusion CNNs [7], which use a VGG16 deep architecture on the UCF-101 and HMDB-51 datasets. We also include some complex network-based methods: ST-ResNet [39] and TSN [8]. Compared with the traditional hand-crafted methods and deep learning methods, our proposed method achieves better performance.

5. Conclusion

In this paper, we proposed an action recognition framework by utilizing frame level deep features of the CNNs and processing them through encode-decode architecture. Firstly, videos are split into video clips and frames, which are fed to 3DCNNs and 2DCNNs respectively. This help in recognizing spatio-temporal features and mining the relationship in frames. After encoding, we employ bi-direction LSTM and hierarchical structure to process these features, meanwhile, the soft-attention mechanism is embedded to these layers to automatically find the region of interesting. When it comes to decoder, the temporal attention is used to refine the input features and output the final action category. Due to represent features effectively and mine the important relationship, our proposed method achieves the state-of-the-art result.

In the future work, we pay more attention to encode-decode architecture and we employ popular methods into our framework.

Acknowledgments

This work was primarily supported by National Natural Science Foundation of China (NSFC) with grant number 6117019.

References

[1] S. Herath, M. Harandi and F. Porikli, Going deeper into action recognition: A survey, *Image and Vision Computing* **60** (2017), 4–21.

[2] S. Sharma, R. Kiros and R. Salakhutdinov, Action recognition using visual attention, *ICLR Workshop* (2016).

[3] S. Ji, W. Xu, M. Yang and K. Yu, 3d convolutional neural networks for human action recognition, *IEEE Transactions on Pattern Analysis Machine Intelligence* **35**(1) (2012), 221–231.

[4] J. Carreira and A. Zisserman, Quo vadis, action recognition? A new model and the kinetics dataset, in *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR) IEEE*, 2017, pp. 4724–4733.

[5] D. Tran, L.D. Bourdev, R. Fergus, L. Torresani and M. Paluri, Learning spatiotemporal features with 3d convolutional networks, *IEEE International Conference on Computer Vision (ICCV)*, 2015, pp. 4489–4497.

[6] K. Simonyan and A. Zisserman, Two-stream convolutional networks for action recognition in videos, *Advances in Neural Information Processing Systems* **1**(4) (2014), 568–576.

[7] C. Feichtenhofer, A. Pinz and A. Zisserman, Convolutional two-stream network fusion for video action recognition, *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 1933–1941.

[8] L. Wang, Y. Xiong, Z. Wang, Y. Qiao, D. Lin, X. Tang and L.V. Gool, Temporal segment networks: Towards good practices for deep action recognition, *Acm Transactions on Information Systems* **22**(1) (2016), 20–36.

[9] S. Hochreiter and J. Schmidhuber, Long short-term memory, *Neural Computation* **9**(8) (1997), 1735–1780.

[10] A. Graves, N. Jaitly and A.-R. Mohamed, Hybrid speech recognition with deep bidirectional lstm, *in Automatic Speech Recognition and Understanding (ASRU), 2013 IEEE Workshop on IEEE*, 2014, pp. 273–278.

[11] I. Sutskever, O. Vinyals and Q.V. Le, Sequence to sequence learning with neural networks, *in Advances in Neural Information Processing Systems*, 2014, pp. 3104–3112.

[12] L. Yao, A. Torabi, K. Cho, N. Ballas, C. Pal, H. Larochelle and A.C. Courville, Describing videos by exploiting temporal structure, *Proceedings of the IEEE International Conference on Computer Vision, ICCV*, 2015, pp. 4507–4515.

[13] Z. Li, K. Gavrilyuk, E. Gavves, M. Jain and C.G. Snoek, Videolstm convolves, attends and flows for action recognition, *Computer Vision and Image Understanding* **166** (2018), 41–50.

[14] K. Cho, A. Courville and Y. Bengio, Describing multimedia content using attention-based encoder-decoder networks, *IEEE Transactions on Multimedia* **17**(11) (2015), 1875–1886.

[15] V. Mnih, N. Heess, A. Graves and K. Kavukcuoglu, Recurrent models of visual attention, *Advances in Neural Information Processing Systems* **3** (2014), 2204–2212.

[16] H. Yang, J. Zhang and S. Li, Attend it again: Recurrent attention convolutional neural network for action recognition, *Applied Science* (2018).

[17] H. Kuehne, H. Jhuang, E. Garrote, T. Poggio and T. Serre, Hmdb: A large video database for human motion recognition, *in IEEE International Conference on Computer Vision, ICCV 2011*, Barcelona, Spain, November, 2011, pp. 2556–2563.

[18] K. Soomro, A.R. Zamir and M. Shah, Ucf101: A dataset of 101 human actions classes from videos in the wild, *Computer Science* (2012).

[19] A. Karpathy, G. Toderici, S. Shetty, T. Leung, R. Sukthankar and F.F. Li, Large-scale video classification with convolutional neural networks, *in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014, pp. 1725–1732.

[20] K. Hara, H. Kataoka and Y. Satoh, Can spatiotemporal 3d cnns retrace the history of 2d cnns and imagenet? *in IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

[21] D.N.T. How, K.S.M. Sahari, H. Yuhuang and L.C. Kiong, Multiple sequence behavior recognition on humanoid robot using long short-term memory (lstm), *in Robotics and Manufacturing Automation (ROMA), 2014 IEEE International Symposium on IEEE*, 2014, pp. 109–114.

[22] Y. Cui, S. Wang, J. Li and Y. Wang, Lstm neural reordering feature for statistical machine translation, *Computer Science* **2** (2016).

[23] S. Wang and J. Jiang, Learning natural language inference with lstm, *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics*, 2015, p. 12.

[24] J. Donahue, L.A. Hendricks, S. Guadarrama, M. Rohrbach, S. Venugopalan, T. Darrell and K. Saenko, Long-term recurrent convolutional networks for visual recognition and description, *in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 677–691.

[25] J.Y. Ng, M.J. Hausknecht, S. Vijayanarasimhan, O. Vinyals, R. Monga and G. Toderici, Beyond short snippets: Deep networks for video classification, *IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 4694–4702.

[26] L. Sun, K. Jia, K. Chen, D. Yan Yeung, B. Shi and S. Savarese, Lattice long short-term memory for human action recognition, arXiv preprint arXiv:1708.03958, 08 2017.

[27] A. Ullah, J. Ahmad, K. Muhammad, M. Sajjad and S.W. Baik, Action recognition in video sequences using deep bi-directional lstm with cnn features, *IEEE Access* **PP**(99) (2017), 1–1.

[28] H. Wang and C. Schmid, Action recognition with improved trajectories, *in IEEE International Conference on Computer Vision (ICCV) IEEE*, 2013, pp. 3551–3558.

[29] F. Baradel, C. Wolf and J. Mille, Pose-conditioned spatiotemporal attention for human action recognition, arXiv preprint arXiv:1703.10106, 2017.

[30] K. Xu, J. Ba, R. Kiros, K. Cho, A. Courville, R. Salakhutdinov, R. Zemel and Y. Bengio, Show, attend and tell: Neural image caption generation with visual attention, *Computer Science* (2015), 2048–2057.

[31] B. Sriman and L. Schomaker, Object attention patches for text detection and recognition in scene images using sift, *in ICPRAM (1)*, 2015, pp. 304–311.

[32] S. Yeung, O. Russakovsky, J. Ning, M. Andriluka, G. Mori and F.F. Li, Every moment counts: Dense detailed labeling of actions in complex videos, *International Journal of Computer Vision (IJCV)* **8** (2015), 1–15.

[33] N. Srivastava, E. Mansimov and R. Salakhudinov, Unsupervised learning of video representations using lstms, *International Conference on Machine Learning*, 2015, pp. 843–852.

[34] K. He, X. Zhang, S. Ren and J. Sun, Deep residual learning for image recognition, *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[35] A. Graves, A.R. Mohamed and G. Hinton, Speech recognition with deep recurrent neural networks, *in IEEE International Conference on Acoustics, Speech and Signal Processing*, 2013, pp. 6645–6649.

[36] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, et al., Tensorflow: Large-scale machine learning on heterogeneous distributed systems, arXiv: Distributed, Parallel, and Cluster Computing, 2016.

[37] L. Sun, K. Jia, D.Y. Yeung and B.E. Shi, Human action recognition using factorized spatio-temporal convolutional networks, *in IEEE International Conference on Computer Vision (ICCV)*, 2015, pp. 4597–4605.

[38] L. Wang, Y. Qiao and X. Tang, Action recognition with trajectory-pooled deep-convolutional descriptors, in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 4305–4314.

[39] C. Feichtenhofer, A. Pinz and R.P. Wildes, Spatiotemporal multiplier networks for video action recognition, *in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) IEEE*, 2017, pp. 7445–7454.

[40] G. Csurka, C.R. Dance, L. Fan, J. Willamowski and C. Bray, Visual categorization with bags of keypoints, *Workshop on Statistical Learning in Computer Vision Eccv* **44**(247) (2004), 1–22.

[41] B. Fernando, S. Gavves, O. Mogrovejo, J. Antonio, A. Ghodrati and T. Tuytelaars, Modeling video evolution for action recognition, *in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 5378–5387.

[42] B. Ni, P. Moulin, X. Yang and S. Yan, Motion part regularization: Improving action recognition via trajectory selection, *in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 3698–3706.

[43] B. Zhang, L. Wang, Z. Wang, Y. Qiao and H. Wang, Realtime action recognition with enhanced motion vector cnns, *in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) IEEE*, 2016, pp. 2718–2726.